COURSE SLIDES · WSQ

Build a Human-AI Workforce with Autonomous AI Agents

WSQ Course Code: TGS-2024043854

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v1.2 · 14 July 2026

COURSE ADMINISTRATION

Welcome & Housekeeping

Digital Attendance (Mandatory)

1. It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
2. The trainer/administrator displays the digital attendance QR code from the SSG portal.
3. Scan the QR code with your mobile phone camera and submit your attendance.
4. A minimum of 75% attendance is required to be eligible for assessment and funding.

About the Trainer

Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

YOUR TRAINER

About the Trainer

Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte. Ltd.

ROLE
Principal Trainer, Tertiary Infotech Academy Pte. Ltd.

CERTIFICATION
AI, automation and cloud certified — building autonomous AI agents and agentic workflows.

DELIVERS
WSQ courses on AI agents, automation, data engineering and cloud.

FOUNDER
Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

ICE-BREAKER

Let's Know Each Other

1. Your name and organisation / role.

2. Your experience with AI, automation or coding (if any).

3. What you want to be able to automate with AI agents after this course.

HOUSEKEEPING

Ground Rules

1. Set your mobile phone to silent mode.
2. Participate actively — no question is too small.
3. Mutual respect: agree to disagree.
4. One conversation at a time.
5. Be punctual; return from breaks on time.
6. 75% attendance is required.

COURSE PORTAL

Download Course Material

1. Sign in at the LMS-TMS portal
2. Open this course from your dashboard
3. Go to the Courseware tab
4. Download the Slides (PPT/PDF), Learner Guide and Lesson Plan
5. Keep them open for the open-book assessment

Portal: https://lms-tms.tertiaryinfotech.com

SCHEDULE

Lesson Plan — 2 Days, 8 hours/day

Day 1

- Day 1 — Hermes Agent (Topic 1) & OpenClaw (Topic 2)
  - Topic 1: Hermes Agent (Labs 1–14)
  - Topic 2: OpenClaw (Labs 15–26)

Day 2

- Day 2 — Paperclip (Topic 3) & Final Assessment
  - Topic 3: Paperclip (Labs 27–34)
  - Final Assessment (WA + PP)
- Daily timing
  - 9:30am–6:30pm · 1-hour lunch · tea breaks within

WHAT YOU'LL ACHIEVE

Learning Outcomes

1

Analyze AI Applications

Analyse AI (LLM-based) applications across a range of industries to identify their capabilities and limitations.

2

Design & Chatbot Efficiency

Establish the relationship between AI algorithm/agent design and chatbot/agent efficiency.

3

Evaluate & Improve RAG

Evaluate and improve the effectiveness of AI (RAG and agent) applications in product development.

Briefing for Assessment

1. Place phones and other materials under the table or on the floor.
2. No photos or recording of assessment scripts.
3. No discussion during the assessment.
4. Use a black/blue pen for hard-copy assessments.
5. No liquid paper / correction tape.
6. Scripts are collected when time is up.

FINAL ASSESSMENT

Assessment

1. Written Assessment (WA) — Short-Answer Questions (SAQ), 1 hour, open book.
2. Practical Performance (PP) — hands-on agent-building tasks that follow the course labs, 1 hour, open book.
3. Format: Open Book — slides, Learner Guide and approved materials only.
4. A minimum of 75% attendance is required to be eligible for assessment and funding.
5. An appeal process is available if required.

ON ASSESSMENT DAY

Assessment Flow

1. TRAQOM survey — scan the QR code on the LMS
2. Assessment digital attendance — scan the SSG QR
3. Sit WA (SAQ) then PP — open book
4. Submit your answers on the LMS
5. Sign the Assessment Summary Record

CORE CONCEPTS

Autonomous AI Agents Fundamentals

OVERVIEW

What is an Autonomous AI Agent?

1

Perceive → reason → act → remember

An LLM-driven loop: it takes in a request, decides what to do, acts, then remembers the outcome.

2

Uses tools & skills

It calls web search, images, email, code and integrations — not just chat — to get real work done.

3

Runs across channels

The same agent is reachable on Telegram, WhatsApp, Discord, Slack and the terminal.

4

Autonomy under oversight

It works on its own, but important actions pause for human approval.

THE BUILDING BLOCKS

The Agent Stack

1

Model / LLM

The reasoning engine — Claude, OpenAI, OpenRouter, MiniMax or DeepSeek.

2

Memory

Durable, user- or company-scoped recall across sessions and surfaces.

3

Skills & Tools

Reusable capabilities plus tool and third-party integration calls.

4

Channels

Where the agent meets people: Telegram, WhatsApp, Discord, Slack.

5

Scheduler & Security

Crons and a heartbeat automate work; sandboxes, budgets and approvals contain it.

HUMAN-IN-THE-LOOP GOVERNANCE

Powerful, but always under human oversight.

Approvals gate risky actions, budgets cap spend (warn at 80%, pause at 100%), and sandboxes isolate execution — so an agent stays safe as it scales.

WHAT YOU'LL BUILD

Three Platforms, One Skillset

Hermes Agent

- Your personal chief-of-staff
- Lives across your messaging apps
- Cross-channel, memory, crons

OpenClaw

- Automates a business back-office
- Node.js gateway daemon
- Skills, tools, nightly reports

Paperclip

- A company of AI agents you govern
- Self-hosted with Docker
- Hiring, budgets, audit trail

HOW EVERY LAB PROGRESSES

The Build Arc

1. Install the runtime → connect a model → add channels → give it memory.
2. Extend it with skills → wire in tools and integrations → schedule work with crons.
3. Secure it with sandboxing, secrets and approvals → drive it through commands.
4. Finish each platform by orchestrating a multi-agent team for an end-to-end outcome.
5. The same arc repeats on Hermes, OpenClaw and Paperclip — so the skillset transfers.

01

TOPIC 01

Hermes Agent — Your Autonomous Personal Chief-of-Staff

Install & Setup · Deployment · Memory & Plugins · Skills · Providers & Model · MCP & Tools · Crons · Subagents · Profile & Kanban · Security

EXAM WEIGHTING 33%

Key Concepts — Hermes Agent — Your Autonomous Personal Chief-of-Staff

1. Hermes Agent (Nous Research) installs on your own machine — via the install script or the Hermes Desktop app — and requires a model with a >=64,000-token context window.

2. Memory and plugins make the agent stateful and extensible: a three-layer memory (agent-curated notes, FTS5 cross-session recall, Honcho user modelling) plus installable skills and MCP tools.

3. Providers & models are swappable with no lock-in (Nous Portal, OpenRouter, OpenAI, any endpoint); MCP servers and the Tool Gateway (web search, image, TTS) give the agent real-world reach.

4. Cron jobs automate recurring work; subagents & delegation split work across isolated backends; the profile and Kanban board let you supervise the agent's tasks — all under security controls.

WHAT YOU'LL DO

Hands-On Labs — Hermes Agent — Your Autonomous Personal Chief-of-Staff

Labs 1–5

- Install and Setup Hermes
- Deployment — Local Install & Hermes Desktop App
- Memory and Plugins
- Skills
- Providers & Model

Labs 6–10

- MCP and Tools
- Cron Jobs & Automation
- Subagents & Delegation
- Profile and Kanban
- Security

Labs 11–14

- Use Case — Make a Video with Hyperframe
- Visualization
- Build a Multi-Agent Workflow
- Webhook

TOPIC 01 · HANDS-ON

Install and Setup Hermes

LAB 1

Install Hermes Agent on your laptop using any of the official install methods (Desktop installer, install script, PowerShell, from source, or Nix), run the first-time setup wizard, and confirm a healthy install. This is the foundation for the Athena chief-of-staff agent you build across the track. Prerequisite: Git; the installer auto-handles Python 3.11, Node.js v22, ripgrep and ffmpeg.

You'll build

A working Hermes Agent install that passes 'hermes doctor' and opens the TUI.

Tools: Hermes Agent, Nous Portal, terminal

LAB 1

Install and Setup Hermes

STEP 1 OF 8

1

Install on macOS / Linux / WSL2 — run the official install script

```
$ curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash
```

LAB 1

Install and Setup Hermes

STEP 2 OF 8

2

Install on Windows — run the PowerShell installer

```
$ iex (irm https://hermes-agent.nousresearch.com/install.ps1)
```

LAB 1

Install and Setup Hermes

STEP 3 OF 8

3

Install Hermes Desktop — download the installer from hermes-agent.nousresearch.com, or launch it from an existing CLI install

```
$ hermes desktop
```

LAB 1

Install and Setup Hermes

STEP 4 OF 8

4

Other install options: from source (Development Setup) or Nix/NixOS (flake + module)

LAB 1

Install and Setup Hermes

STEP 5 OF 8

5

Reload your shell so the 'hermes' command is on PATH

```
$ source ~/.zshrc
```

LAB 1

Install and Setup Hermes

STEP 6 OF 8

6

Run the first-time setup wizard

```
$ hermes setup --portal
```

LAB 1

Install and Setup Hermes

STEP 7 OF 8

7

Check the install is healthy

```
$ hermes doctor
```

LAB 1

Install and Setup Hermes

STEP 8 OF 8

8

Launch the agent and say hello

```
$ hermes --tui
```

LAB 1 · VERIFY

Install and Setup Hermes

✅ Test it

'hermes doctor' reports all green and the TUI opens and replies to a message.

Deployment — Local Install & Hermes Desktop App

LAB 2

Run Hermes as a persistent local deployment (preferred) and also install the Hermes Desktop app for a GUI experience. Confirm the local gateway is serving and keep the runtime up to date.

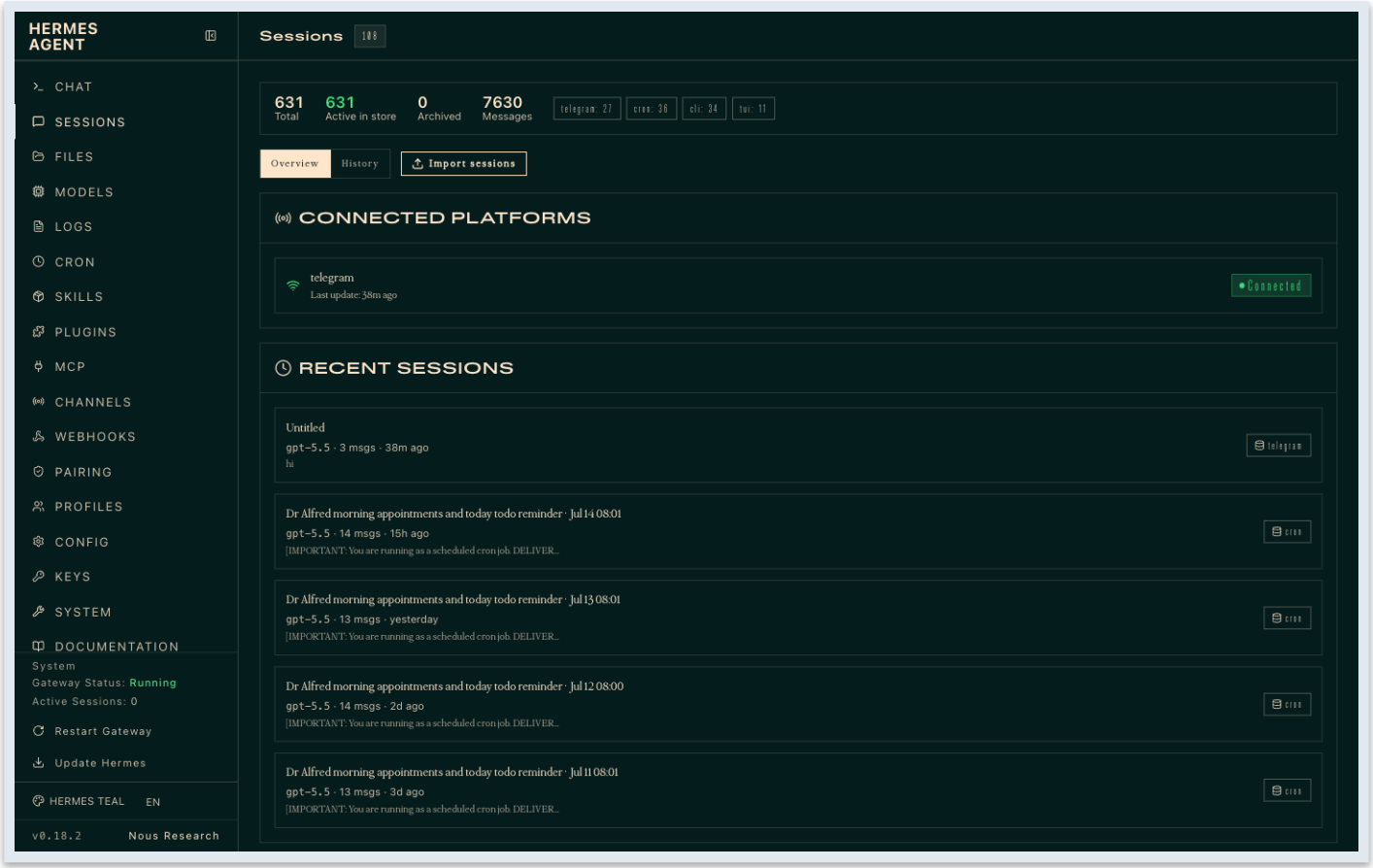
You'll build

A local Hermes deployment plus the Desktop app, both talking to the same agent.

Tools: Hermes Agent, Hermes Desktop, terminal

LAB 2 · LIVE SCREENSHOT

Hermes in the Browser — Dashboard



The Hermes Dashboard (http://127.0.0.1:9119) — your local deployment's sessions view

LAB 2

Deployment — Local Install & Hermes Desktop App

STEP 1 OF 5

1

Confirm the local install runs (preferred deployment)

```
$ hermes --version
```

LAB 2

Deployment — Local Install & Hermes Desktop App

STEP 2 OF 5

2

Download and install the Hermes Desktop app

LAB 2

Deployment — Local Install & Hermes Desktop App

STEP 3 OF 5

3

Open the Desktop app and sign in with the same Nous Portal account

LAB 2

Deployment — Local Install & Hermes Desktop App

STEP 4 OF 5

4

Verify the local gateway is serving

```
$ hermes gateway status
```

Deployment — Local Install & Hermes Desktop App

STEP 5 OF 5

5

Keep the runtime up to date

```
$ hermes update
```

LAB 2 · VERIFY

Deployment — Local Install & Hermes Desktop App

✅ Test it

Both the local CLI and the Desktop app drive the same agent; 'hermes gateway status' is healthy.

TOPIC 01 · HANDS-ON

Memory and Plugins

LAB 3

Explore Hermes' three-layer memory (agent-curated notes, FTS5 cross-session recall, Honcho user modelling). Teach Athena a preference, recall it in a fresh session, and enable a plugin to extend behaviour.

You'll build

Athena remembers a stated preference across sessions and runs an enabled plugin.

Tools: Hermes Agent, memory, plugins

LAB 3

Memory and Plugins

STEP 1 OF 4

1

In a session, teach Athena a durable preference

LAB 3

Memory and Plugins

STEP 2 OF 4

2

Start a new session and confirm the preference is recalled

```
$ hermes --continue
```

LAB 3

Memory and Plugins

STEP 3 OF 4

3

Inspect stored memory/sessions

```
$ hermes sessions list
```

LAB 3

Memory and Plugins

STEP 4 OF 4

4

Enable a plugin to extend the agent

LAB 3 · VERIFY

Memory and Plugins

✅ Test it

A preference taught in one session is recalled in a new session, and the enabled plugin is active.

TOPIC 01 · HANDS-ON

Skills

LAB 4

Browse and install open-standard skills (agentskills.io / Skills Hub) so Athena can perform new tasks. Skills are portable and can self-improve as the agent uses them.

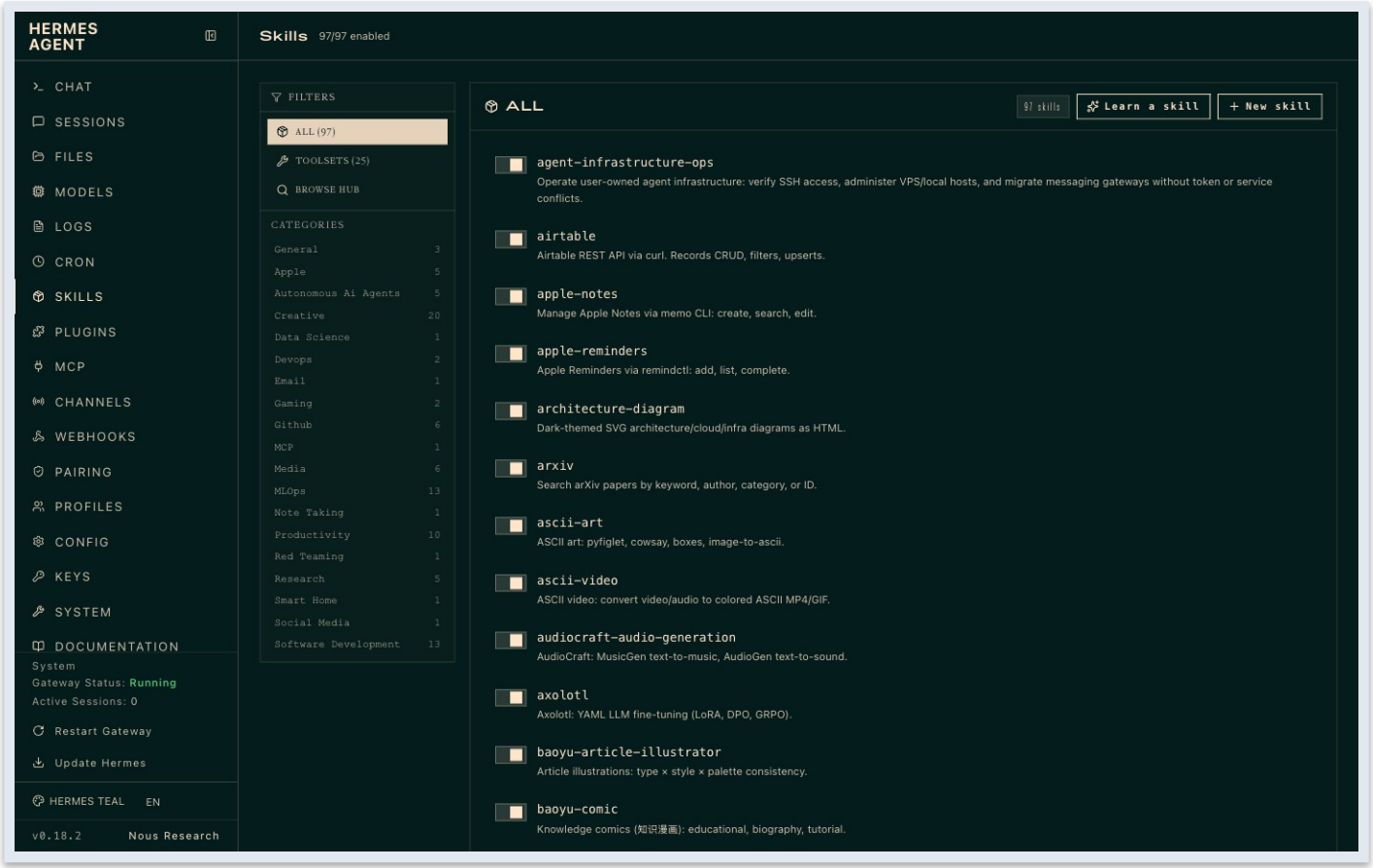
You'll build

Athena equipped with at least one installed skill that it can invoke on request.

Tools: Hermes Agent, agentskills.io, Skills Hub

LAB 4 · LIVE SCREENSHOT

Hermes in the Browser — Skills



The Skills page of a live Hermes install — 97 skills, filterable by category, plus Learn-a-skill and New-skill

FROM A LIVE HERMES INSTALL · 97 SKILLS

Hermes Skill Categories (1 of 2)

1. **Creative**
   20 skills · e.g. architecture-diagram, ascii-art, ascii-video
2. **Mlops**
   13 skills · e.g. evaluating-llms-harness, weights-and-biases, huggingface-hub
3. **Software Development**
   13 skills · e.g. debugging-hermes-tui-commands, hermes-agent-skill-authoring, hermes-s6-container-supervision
4. **Productivity**
   10 skills · e.g. airtable, google-workspace, linear
5. **Github**
   6 skills · e.g. codebase-inspection, github-auth, github-code-review
6. **Media**
   6 skills · e.g. gif-search, heartmula, songsee
7. **Apple**
   5 skills · e.g. apple-notes, apple-reminders, findmy
8. **Autonomous Ai Agents**
   5 skills · e.g. claude-code, codex, hermes-agent
9. **Research**
   5 skills · e.g. arxiv, blogwatcher, llm-wiki
10. **Other**
    3 skills · e.g. computer-use, dogfood, yuanbao

FROM A LIVE HERMES INSTALL · 97 SKILLS

Hermes Skill Categories (2 of 2)

1. **Devops**
   2 skills · e.g. agent-infrastructure-ops, webhook-subscriptions
2. **Gaming**
   2 skills · e.g. minecraft-modpack-server, pokemon-player
3. **Data Science**
   1 skill · e.g. jupyter-live-kernel
4. **Email**
   1 skill · e.g. himalaya
5. **Mcp**
   1 skill · e.g. native-mcp
6. **Note Taking**
   1 skill · e.g. obsidian
7. **Red Teaming**
   1 skill · e.g. godmode
8. **Smart Home**
   1 skill · e.g. openhue
9. **Social Media**
   1 skill · e.g. xurl

LAB 4

Skills

STEP 1 OF 4

1

Browse the skills hub

```
$ hermes skills browse
```

LAB 4

Skills

STEP 2 OF 4

2

Search for a skill by keyword

```
$ hermes skills search <keyword>
```

LAB 4

Skills

STEP 3 OF 4

3

Install a skill

```
$ hermes skills install <owner/skills/name>
```

LAB 4

Skills

STEP 4 OF 4

4

Ask the agent to use the new skill and observe self-improvement

LAB 4 · VERIFY

Skills

✅ Test it

The installed skill appears in the agent's toolset and runs when invoked.

TOPIC 01 · HANDS-ON

Providers & Model

LAB 5

Connect a model provider (Nous Portal, OpenRouter, OpenAI or any endpoint) and select a model that meets the >=64,000-token context requirement. Switch providers to compare speed and quality.

You'll build

Athena running on a chosen provider/model, with the ability to switch on demand.

Tools: Hermes Agent, Nous Portal, OpenRouter

LAB 5

Providers & Model

STEP 1 OF 4

1

Open the interactive provider/model picker

```
$ hermes model
```

LAB 5

Providers & Model

STEP 2 OF 4

2

Set a model explicitly

```
$ hermes config set model anthropic/claude-opus-4.6
```

LAB 5

Providers & Model

STEP 3 OF 4

3

Add a provider key if using a cloud endpoint

```
$ hermes config set OPENROUTER_API_KEY sk-or-...
```

LAB 5

Providers & Model

STEP 4 OF 4

4

Confirm the active model meets the >=64k context rule

```
$ hermes model
```

LAB 5 · VERIFY

Providers & Model

✅ Test it

The agent starts on the selected model and can be switched to another provider without reinstalling.

TOPIC 01 · HANDS-ON

MCP and Tools

LAB 6

Add Model Context Protocol (MCP) servers in the Hermes config and use the bundled Tool Gateway (web search, image generation, TTS) so Athena can act on the outside world.

You'll build

Athena wired to an MCP server (e.g. GitHub) and able to use a Tool Gateway tool.

Tools: Hermes Agent, MCP, Tool Gateway

LAB 6

MCP and Tools

STEP 1 OF 4

1

Open the Hermes config to add an MCP server

LAB 6

MCP and Tools

STEP 2 OF 4

2

Add a GitHub MCP server entry

```
$ npx -y @modelcontextprotocol/server-github
```

LAB 6

MCP and Tools

STEP 3 OF 4

3

Restart so the MCP server is picked up, then verify

```
$ hermes doctor
```

LAB 6

MCP and Tools

STEP 4 OF 4

4

Use a Tool Gateway tool (web search / image / TTS) from a chat

LAB 6 · VERIFY

MCP and Tools

✅ Test it

The agent lists the new MCP server's tools and completes a task using a Tool Gateway tool.

TOPIC 01 · HANDS-ON

Cron Jobs & Automation

LAB 7

Schedule Athena to run recurring jobs — for example an 8am daily briefing sent to your messaging app — and verify the scheduled run fires and produces output.

You'll build

A scheduled job (e.g. a daily briefing) that runs automatically and delivers a result.

Tools: Hermes Agent, scheduler/crons

LAB 7

Cron Jobs & Automation

STEP 1 OF 4

1

Define a recurring job (daily 8am briefing) in the Hermes automation/cron config

LAB 7

Cron Jobs & Automation

STEP 2 OF 4

2

Confirm the automation is registered

```
$ hermes doctor
```

LAB 7

Cron Jobs & Automation

STEP 3 OF 4

3

Trigger the job once to confirm it produces output

LAB 7

Cron Jobs & Automation

STEP 4 OF 4

4

Confirm the next scheduled run is queued

LAB 7 · VERIFY

Cron Jobs & Automation

✅ Test it

The scheduled job runs at its time (or on manual trigger) and delivers the expected briefing.

TOPIC 01 · HANDS-ON

Subagents & Delegation

LAB 8

Turn Athena into a coordinator that delegates tasks to worker subagents running on isolated terminal backends (local, docker, ssh, modal), then aggregates their results.

You'll build

A coordinator agent that delegates a task to one or more worker subagents and combines the output.

Tools: Hermes Agent, terminal backends (docker/ssh/modal)

LAB 8

Subagents & Delegation

STEP 1 OF 4

1

Choose an isolated backend for worker agents

```
$ hermes config set terminal.backend docker
```

LAB 8

Subagents & Delegation

STEP 2 OF 4

2

Ask the coordinator to delegate a multi-part task to subagents

LAB 8

Subagents & Delegation

STEP 3 OF 4

3

Observe each subagent run in isolation and report back

LAB 8

Subagents & Delegation

STEP 4 OF 4

4

Review the coordinator's aggregated result

LAB 8 · VERIFY

Subagents & Delegation

✅ Test it

The coordinator delegates to worker subagents on an isolated backend and returns a combined result.

Profile and Kanban

LAB 9

Configure the agent's profile and use the Hermes Kanban board to watch tasks move through todo -> in progress -> done, giving you human oversight of what Athena is working on.

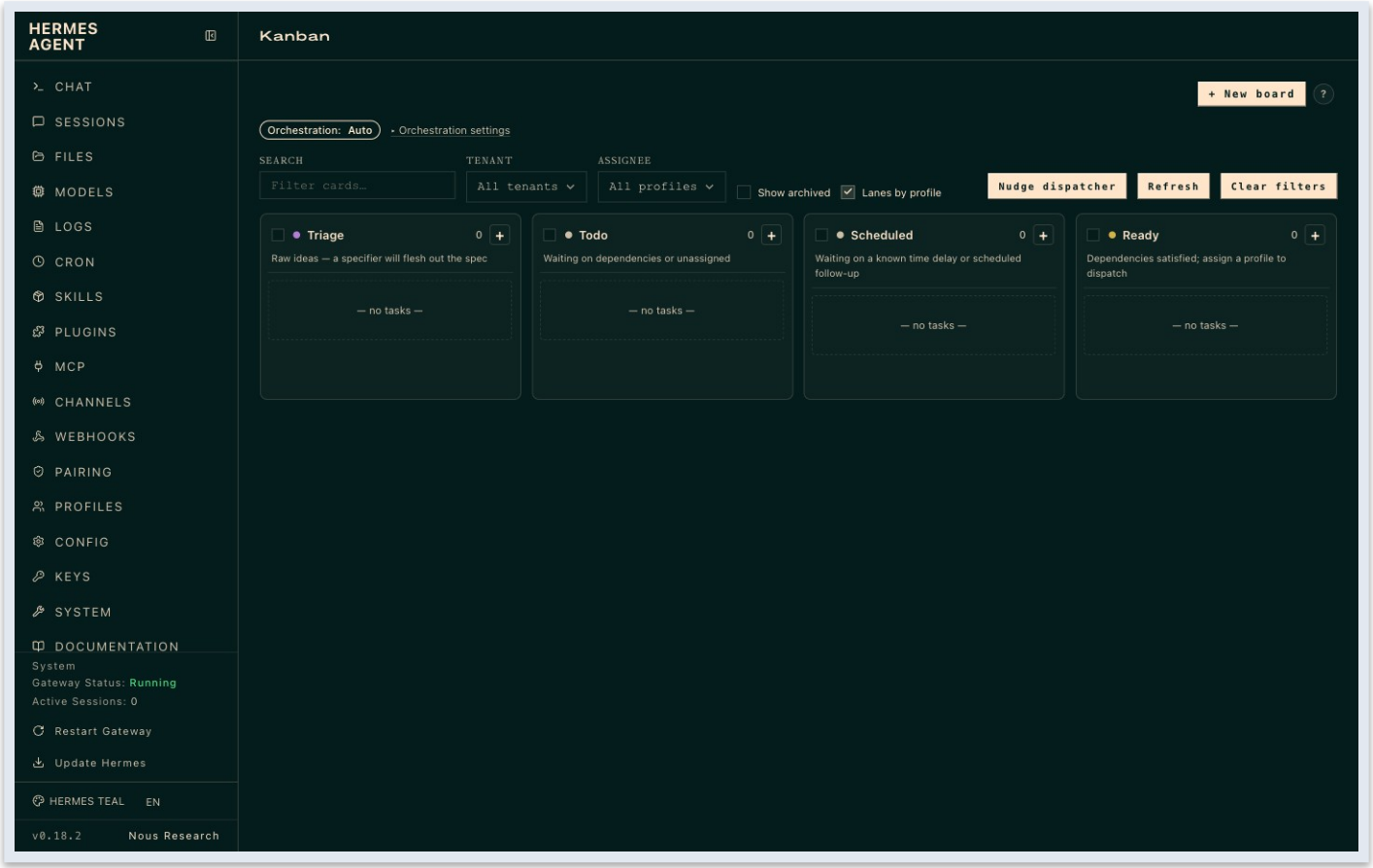
You'll build

A configured agent profile and a Kanban board tracking the agent's live tasks.

Tools: Hermes Agent, profile, Kanban board

LAB 9 · LIVE SCREENSHOT

Hermes in the Browser — Kanban



The Kanban board — supervise the agent's tasks as they move across the columns

LAB 9

Profile and Kanban

STEP 1 OF 4

1

Set up the agent profile (identity, defaults, preferences)

LAB 9

Profile and Kanban

STEP 2 OF 4

2

Open the Kanban board view

LAB 9

Profile and Kanban

STEP 3 OF 4

3

Assign a task and watch it move todo -> in progress -> done

LAB 9

Profile and Kanban

STEP 4 OF 4

4

Use the board to review and accept completed work

LAB 9 · VERIFY

Profile and Kanban

✅ Test it

The profile is set and the Kanban board shows tasks progressing across its columns.

TOPIC 01 · HANDS-ON

Security

LAB 10

Harden Athena: run risky work in an isolated terminal backend, manage secrets safely, and require approval before sensitive actions — applying least privilege throughout.

You'll build

A hardened agent that sandboxes execution, protects secrets and prompts for approval on risky actions.

Tools: Hermes Agent, terminal backends, secrets management

LAB 10

Security

STEP 1 OF 4

1

Isolate execution in a sandboxed backend

```
$ hermes config set terminal.backend docker
```

LAB 10

Security

STEP 2 OF 4

2

Store provider/tool secrets via config rather than plain text

```
$ hermes config set <PROVIDER>_API_KEY <value>
```

LAB 10

Security

STEP 3 OF 4

3

Require approval before the agent runs risky actions

LAB 10

Security

STEP 4 OF 4

4

Apply least privilege — grant only the tools/skills the task needs

LAB 10 · VERIFY

Security

✅ Test it

Risky actions run only in the sandbox and only after approval; secrets are not stored in plain text.

Use Case — Make a Video with Hyperframe

LAB 11

Put Athena to work on an end-to-end creative task: use the agent to drive the Hyperframe tool and generate a short video from a concept brief, then review and refine the result.

You'll build

A short video generated end-to-end by the agent using Hyperframe.

Tools: Hermes Agent, Hyperframe

LAB 11

Use Case — Make a Video with Hyperframe

STEP 1 OF 4

1

Connect / set up the Hyperframe video tool for the agent

LAB 11

Use Case — Make a Video with Hyperframe

STEP 2 OF 4

2

Brief the agent with a short video concept (topic, style, length)

LAB 11

Use Case — Make a Video with Hyperframe

STEP 3 OF 4

3

Have the agent generate the video with Hyperframe

LAB 11

Use Case — Make a Video with Hyperframe

STEP 4 OF 4

4

Review the output and refine the brief to regenerate if needed

LAB 11 · VERIFY

Use Case — Make a Video with Hyperframe

✅ Test it

The agent produces a playable video from your brief using Hyperframe.

TOPIC 01 · HANDS-ON

Visualization

LAB 12

Use Hermes' visualization capability to see the agent's workflow, task flow and data at a glance, making the agent's behaviour transparent and easier to supervise.

You'll build

A visualization of the agent's workflow / activity you can read and interpret.

Tools: Hermes Agent, visualization

LAB 12

Visualization

STEP 1 OF 3

1

Open the visualization view for the agent's activity

LAB 12

Visualization

STEP 2 OF 3

2

Generate a visualization of a workflow or dataset

LAB 12

Visualization

STEP 3 OF 3

3

Interpret the visualization to understand what the agent did

LAB 12 · VERIFY

Visualization

✅ Test it

A visualization of the agent's workflow/data renders and communicates the agent's activity.

TOPIC 01 · HANDS-ON

Build a Multi-Agent Workflow

LAB 13

Design a multi-agent workflow in which several agents take on distinct roles and hand off tasks to one another to complete a larger goal, building on the subagents and delegation concepts from earlier labs.

You'll build

A working multi-agent workflow where agents hand off tasks to complete a goal.

Tools: Hermes Agent, subagents, workflow

LAB 13

Build a Multi-Agent Workflow

STEP 1 OF 4

1

Define the agents and their roles in the workflow

LAB 13

Build a Multi-Agent Workflow

STEP 2 OF 4

2

Connect the agents so outputs hand off between them

LAB 13

Build a Multi-Agent Workflow

STEP 3 OF 4

3

Run the workflow end-to-end on a sample goal

LAB 13

Build a Multi-Agent Workflow

STEP 4 OF 4

4

Verify each stage completed and the final result is correct

LAB 13 · VERIFY

Build a Multi-Agent Workflow

✅ Test it

The multi-agent workflow runs end-to-end with each agent handing off to the next and producing the final result.

TOPIC 01 · HANDS-ON

Webhook

LAB 14

Wire the agent to the outside world with webhooks: trigger the agent from an incoming webhook payload and/or have it call an outgoing webhook, then secure the endpoint.

You'll build

A webhook that triggers the agent (or that the agent calls), verified end-to-end.

Tools: Hermes Agent, webhooks

LAB 14

Webhook

STEP 1 OF 4

1

Configure a webhook trigger/endpoint for the agent

LAB 14

Webhook

STEP 2 OF 4

2

Send a test payload to the webhook

LAB 14

Webhook

STEP 3 OF 4

3

Confirm the agent receives the event and responds/acts

LAB 14

Webhook

STEP 4 OF 4

4

Secure the webhook (secret/token, allowlist)

LAB 14 · VERIFY

Webhook

✅ Test it

A test payload to the webhook triggers the agent and produces the expected action, with the endpoint secured.

TOPIC RECAP

Recap — Hermes Agent — Your Autonomous Personal Chief-of-Staff

1. You can now: LO1: Install and configure Hermes Agent on a local machine.
2. You can now: LO1: Deploy Hermes locally and run it through the Hermes Desktop app.
3. You can now: LO2: Give the agent persistent memory and extend it with plugins.
4. You can now: LO3: Extend the agent with installable, self-improving skills.
5. You can now: LO2: Connect an LLM provider and switch models with no lock-in.
6. You can now: LO3: Give the agent real-world reach with MCP servers and the Tool Gateway.

02

TOPIC 02

OpenClaw — Automating a Business Back-Office

Install · Providers · Channels · Skills · Tools · Commands · Crons · Memory · Dreaming · Security · Multi-Agent · Use Cases

EXAM WEIGHTING 33%

Key Concepts — OpenClaw — Automating a Business Back-Office

1. OpenClaw runs on Node.js 24 LTS as a long-running gateway daemon that hosts every channel, tool, cron and skill for the Nimbus Supplies back office.

2. Providers are connected by OAuth sign-in (OpenAI Codex, MiniMax) or API key (Anthropic, DeepSeek, OpenRouter); skills from skills.sh and integrations such as Firecrawl and AgentMail extend the agent.

3. 'Dreaming' lets the agent reflect during idle time — reviewing memory and generating follow-ups — while profiles and allow/deny lists keep every channel least-privileged and audited.

4. Crons schedule recurring work, a heartbeat auto-restarts the gateway, and a multi-agent capstone splits work across cooperating Sales, Research and Ops agents for real back-office use cases.

WHAT YOU'LL DO

Hands-On Labs — OpenClaw — Automating a Business Back-Office

Labs 15–18

- Install OpenClaw
- Models & Providers
- Channels
- Skills

Labs 19–22

- Tools & Integrations
- Commands
- Cron Jobs & Heartbeat
- Memory & Context

Labs 23–26

- Security
- Multi-Agent Workforce
- Dreaming — Idle Reflection & Self-Improvement
- Use Cases & Key Functions

TOPIC 02 · HANDS-ON

Install OpenClaw

LAB 15

The learner installs the OpenClaw platform on their own machine (or a cloud VPS) by first installing Node.js 24 LTS, then installing OpenClaw via npm and running the onboarding wizard, and verifies the install with doctor and gateway status. This gateway becomes the back-office brain of the Nimbus Supplies running example.

You'll build

A running OpenClaw gateway daemon (local or VPS) verified with openclaw doctor

Tools: OpenClaw, Node.js 24 LTS, npm, openclaw gateway

LAB 15

Install OpenClaw

STEP 1 OF 6

1

Install Node.js 24 LTS on macOS with Homebrew and verify

```
$ brew install node@24
```

LAB 15

Install OpenClaw

STEP 2 OF 6

2

Install OpenClaw globally via npm

```
$ npm install -g openclaw@latest
```

LAB 15

Install OpenClaw

STEP 3 OF 6

3

Run the interactive onboarding wizard that creates ~/.openclaw and the gateway

```
$ openclaw onboard
```

LAB 15

Install OpenClaw

STEP 4 OF 6

4

Verify the install version

```
$ openclaw --version
```

LAB 15

Install OpenClaw

STEP 5 OF 6

5

Run the health check (Node, gateway daemon, providers, channels)

```
$ openclaw doctor
```

LAB 15

Install OpenClaw

STEP 6 OF 6

6

Confirm the gateway is running, starting it if needed

```
$ openclaw gateway start
```

LAB 15 · VERIFY

Install OpenClaw

✅ Test it

openclaw --version prints a version, openclaw doctor shows green checks for Node 24 and the gateway daemon (provider/channel checks pending until Labs 16-17), and openclaw gateway status reports running.

Models & Providers

LAB 16

The learner connects a language model to OpenClaw, trying the supported provider options — OAuth sign-ins (OpenAI Codex, MiniMax) and API-key providers (Anthropic, DeepSeek, OpenRouter) — learns where credentials are stored under ~/.openclaw/auth/, hot-swaps between models, and runs a Nimbus Supplies smoke test.

You'll build

A Nimbus Supplies agent wired to a working model provider that passes openclaw model test

Tools: OpenClaw, OpenAI Codex / MiniMax (OAuth), Anthropic / DeepSeek / OpenRouter (API key)

LAB 16

Models & Providers

STEP 1 OF 7

1

See the available models and the current selection

```
$ openclaw models list
```

LAB 16

Models & Providers

STEP 2 OF 7

2

Option A - sign in to OpenAI Codex via OAuth

```
$ openclaw models auth login --provider openai-codex
```

LAB 16

Models & Providers

STEP 3 OF 7

3

Option C - set an Anthropic API key and select a Claude model

```
$ openclaw config set model anthropic/claude-opus-4-7
```

LAB 16

Models & Providers

STEP 4 OF 7

4

Test that the active model responds

```
$ openclaw model test
```

LAB 16

Models & Providers

STEP 5 OF 7

5

Persist an API key so it survives reboots

```
$ echo 'export ANTHROPIC_API_KEY="sk-ant-..."' >> ~/.zshrc
```

LAB 16

Models & Providers

STEP 6 OF 7

6

Hot-swap the global default model and confirm

```
$ openclaw model use deepseek/deepseek-v4
```

LAB 16

Models & Providers

STEP 7 OF 7

7

Give the agent a Nimbus Supplies smoke test from CLI chat

```
$ openclaw
```

LAB 16 · VERIFY

Models & Providers

✅ Test it

openclaw models list shows the connected models, openclaw model current prints the active one, openclaw model test returns a short successful completion with no auth error, and OAuth credentials appear under ~/.openclaw/auth/.

TOPIC 02 · HANDS-ON

Channels

LAB 17

The learner gives customers a way to reach the Nimbus Supplies agent by creating a Telegram bot end-to-end via BotFather, registering it with a token, pairing a WhatsApp number by scanning a QR code, and running both channels at once through a single OpenClaw gateway so the same agent answers on either platform.

You'll build

Telegram and WhatsApp both live on one OpenClaw gateway, answered by the same Nimbus Supplies agent

Tools: OpenClaw, Telegram (BotFather), WhatsApp, openclaw channel

LAB 17

Channels

STEP 1 OF 6

1

Create a Telegram bot in BotFather with /newbot and copy the HTTP API token

LAB 17

Channels

STEP 2 OF 6

2

Register the Telegram channel with the bot token

```
$ openclaw channel add telegram --token 123456789:ABC-DEF1234ghIkl-zyx57W2v1u123ew11
```

LAB 17

Channels

STEP 3 OF 6

3

Start the Telegram channel

```
$ openclaw channel start telegram
```

LAB 17

Channels

STEP 4 OF 6

4

Add the WhatsApp channel to print a pairing QR code

```
$ openclaw channel add whatsapp
```

LAB 17

Channels

STEP 5 OF 6

5

Start WhatsApp, then scan the QR from WhatsApp -> Linked Devices -> Link a Device

```
$ openclaw channel start whatsapp
```

LAB 17

Channels

STEP 6 OF 6

6

Confirm both channels are running simultaneously

```
$ openclaw channel list
```

LAB 17 · VERIFY

Channels

✅ Test it

openclaw channel list shows telegram and whatsapp both running, a message to the Telegram bot gets an agent reply, a message to the linked WhatsApp account gets an agent reply, and openclaw gateway status shows both connected.

Skills

LAB 18

The learner installs and removes skills from the skills.sh registry, invokes them from chat, and builds a custom Nimbus Supplies skill (nimbus-quote) under ~/.openclaw/skills/ so the agent handles a recurring back-office task — drafting an itemised customer quote with GST — the same way every time.

You'll build

Registry skills plus a custom nimbus-quote skill that produces itemised, GST-inclusive quotes

Tools: OpenClaw, skills.sh registry, ~/.openclaw/skills/, openclaw skills

LAB 18

Skills

STEP 1 OF 7

1

List the skills currently installed

```
$ openclaw skills list
```

LAB 18

Skills

STEP 2 OF 7

2

Install a useful skill from the skills.sh registry

```
$ openclaw skills add web-research --source skills.sh
```

LAB 18

Skills

STEP 3 OF 7

3

Invoke a skill from a channel with the /skill slash command

```
$ /skill web-research "Who are the main office-supplies wholesalers in Singapore?"
```

LAB 18

Skills

STEP 4 OF 7

4

Create the folder for a custom Nimbus Supplies skill

```
$ mkdir -p ~/.openclaw/skills/nimbus-quote
```

LAB 18

Skills

STEP 5 OF 7

5

Author skill.md with front-matter (name, description) and the quoting instructions

LAB 18

Skills

STEP 6 OF 7

6

Restart the gateway so the new skill is picked up

```
$ openclaw gateway restart
```

LAB 18

Skills

STEP 7 OF 7

7

Test the custom skill from a channel

```
$ /skill nimbus-quote "Customer: Acme Cafe. 10 reams recycled A4 paper, 5 boxes black pens."
```

LAB 18 · VERIFY

Skills

✅ Test it

openclaw skills list shows the registry skills plus the custom nimbus-quote, /skill web-research runs end-to-end, and /skill nimbus-quote produces an itemised quote with a 9% GST line and standard terms, marking any unknown price as TBC instead of guessing.

Tools & Integrations

LAB 19

The learner extends the Nimbus Supplies agent with two real integrations — Firecrawl (JS-rendered web scraping) and AgentMail (its own inbox/outbox) — then runs an end-to-end back-office task: research a supplier's website prices, draft an itemised quote with the Lab 18 skill, and email it to a customer. Tool profiles and allow/deny lists scope tools per channel.

You'll build

A Firecrawl + AgentMail integrated agent that scrapes supplier prices and emails an itemised quote

Tools: OpenClaw, Firecrawl, AgentMail, openclaw tools

LAB 19

Tools & Integrations

STEP 1 OF 6

1

Inspect the built-in tool groups (fs, web, runtime, media)

```
$ openclaw tools list
```

LAB 19

Tools & Integrations

STEP 2 OF 6

2

Enable Firecrawl with its API key

```
$ openclaw tools enable firecrawl --api-key "$FIRECRAWL_API_KEY"
```

LAB 19

Tools & Integrations

STEP 3 OF 6

3

Enable AgentMail with its API key and inbox address

```
$ openclaw tools enable agentmail --api-key "$AGENTMAIL_API_KEY" --inbox bot@yourname.agentmail.to
```

LAB 19

Tools & Integrations

STEP 4 OF 6

4

Run the end-to-end task from a channel: scrape prices, draft the quote, email it

LAB 19

Tools & Integrations

STEP 5 OF 6

5

Scope tools on a public channel by denying shell exec

```
$ openclaw channel set telegram --deny exec
```

LAB 19

Tools & Integrations

STEP 6 OF 6

6

Allow only the web tool group on that channel

```
$ openclaw channel set telegram --allow group:web
```

LAB 19 · VERIFY

Tools & Integrations

✅ Test it

openclaw tools list --enabled shows firecrawl and agentmail, the agent scrapes a supplier URL and returns structured prices, an AgentMail send arrives in a real inbox, the end-to-end task produces an itemised quote email, and a channel set to --deny exec refuses shell-exec requests.

Commands

LAB 20

The learner becomes fluent in the two ways to drive OpenClaw daily: the CLI commands (config, model, doctor, gateway, channel, tools, skills, cron) and the in-chat slash commands operators and customers use inside a channel, building a personal quick-reference table for running the Nimbus Supplies back office and streaming live gateway logs.

You'll build

A personal quick-reference of the CLI and slash commands used to run Nimbus Supplies day to day

Tools: OpenClaw, openclaw CLI, in-chat slash commands

LAB 20

Commands

STEP 1 OF 6

1

Read and change settings with config

```
$ openclaw config list
```

LAB 20

Commands

STEP 2 OF 6

2

Run the health check with auto-fix

```
$ openclaw doctor --fix
```

LAB 20

Commands

STEP 3 OF 6

3

Control the gateway daemon that runs everything

```
$ openclaw gateway restart
```

LAB 20

Commands

STEP 4 OF 6

4

Get your channel user ID for the Lab 23 allowlist

```
$ /whoami
```

LAB 20

Commands

STEP 5 OF 6

5

Switch the model for one conversation only

```
$ /model deepseek/deepseek-v4
```

LAB 20

Commands

STEP 6 OF 6

6

Watch activity live while you chat

```
$ openclaw gateway logs --follow
```

LAB 20 · VERIFY

Commands

✅ Test it

You can switch the model both from chat (/model, per-conversation) and globally from the CLI (openclaw model use), /whoami returns your platform user ID, openclaw gateway logs --follow streams live activity, and /help lists the available slash commands.

Cron Jobs & Heartbeat

LAB 21

The learner makes Nimbus Supplies run unattended by scheduling recurring tasks with cron (a nightly sales report and a weekly supplier price-check using Firecrawl) and monitoring uptime with the heartbeat, so the gateway self-checks, auto-restarts on failure, and can page an external monitor if it dies.

You'll build

A nightly sales-report cron, a weekly price-check cron, and an enabled self-healing heartbeat

Tools: OpenClaw, openclaw cron, gateway heartbeat, Firecrawl + AgentMail

LAB 21

Cron Jobs & Heartbeat

STEP 1 OF 6

1

Create the nightly sales-report cron posting to Telegram at 21:00

```
$ openclaw cron create --schedule "0 21 * * *" --prompt "Summarise today's Nimbus Supplies customer chats and any quotes sent." --channel telegram --name nightly-sales-report
```

LAB 21

Cron Jobs & Heartbeat

STEP 2 OF 6

2

List and inspect your crons

```
$ openclaw cron list
```

LAB 21

Cron Jobs & Heartbeat

STEP 3 OF 6

3

Fire a cron on demand to test it now, ignoring the schedule

```
$ openclaw cron run nightly-sales-report
```

LAB 21

Cron Jobs & Heartbeat

STEP 4 OF 6

4

Enable the heartbeat self-check at a 60s interval

```
$ openclaw gateway heartbeat enable --interval 60s
```

LAB 21

Cron Jobs & Heartbeat

STEP 5 OF 6

5

Check the heartbeat status

```
$ openclaw gateway heartbeat status
```

LAB 21

Cron Jobs & Heartbeat

STEP 6 OF 6

6

Confirm auto-restart by stopping the gateway and waiting ~60s

```
$ openclaw gateway stop
```

LAB 21 · VERIFY

Cron Jobs & Heartbeat

✅ Test it

openclaw cron list shows nightly-sales-report and weekly-price-check, openclaw cron run delivers a summary to Telegram immediately, openclaw gateway heartbeat status reports healthy, and after openclaw gateway stop the service manager restarts the gateway within about a minute.

TOPIC 02 · HANDS-ON

Memory & Context

LAB 22

The learner gives the Nimbus Supplies agent durable memory so it recalls customer preferences across conversations: using the in-chat /memory command, inspecting where OpenClaw persists state under ~/.openclaw/, teaching a customer's standing preference, clearing the conversation to prove it is real memory, and confirming recall in a later separate chat.

You'll build

An agent that persistently remembers a customer's standing preferences and applies them in a fresh conversation

Tools: OpenClaw, /memory, /clear, ~/.openclaw/ state

LAB 22

Memory & Context

STEP 1 OF 6

1

See what OpenClaw stores on disk

```
$ ls -la ~/.openclaw/
```

LAB 22

Memory & Context

STEP 2 OF 6

2

Inspect your current per-user memory from chat

```
$ /memory
```

LAB 22

Memory & Context

STEP 3 OF 6

3

Teach the agent a durable customer preference in plain language

LAB 22

Memory & Context

STEP 4 OF 6

4

Clear the conversation to prove it is real memory, not just context

```
$ /clear
```

LAB 22

Memory & Context

STEP 5 OF 6

5

Recall in the cleared chat by requesting a quote that needs the remembered facts

LAB 22

Memory & Context

STEP 6 OF 6

6

Back up memory before major changes

```
$ cp -R ~/.openclaw/ ~/openclaw-backup-$(date +%F)/openclaw-state
```

LAB 22 · VERIFY

Memory & Context

✅ Test it

/memory shows the customer preferences after you teach them, after /clear the /memory list still shows them (short-term context cleared, long-term memory retained), and a fresh quote request for the customer automatically applies recycled paper, Tuesday delivery and the correct recipient name without repetition.

Security

LAB 23

The learner hardens the Nimbus Supplies deployment before real customers: storing API keys as environment variables (never in cleartext config), restricting who can DM the bot with allowlists, applying per-channel tool deny lists, tightening the code-execution sandbox (timeout/memory/network), reviewing the audit log, and rotating provider keys.

You'll build

A hardened agent with keys in env, DM allowlists, per-channel deny lists, a capped sandbox and an exported audit log

Tools: OpenClaw, allowlists, tool deny lists, exec/python sandbox, gateway logs

LAB 23

Security

STEP 1 OF 6

1

Restrict who can DM the Telegram bot to known user IDs

```
$ openclaw channel set telegram --allow-users 12345678,87654321 --pair-mode allowlist
```

LAB 23

Security

STEP 2 OF 6

2

Apply a per-channel tool deny list (no shell or destructive file ops)

```
$ openclaw channel set telegram --deny exec,fs.write,fs.delete
```

LAB 23

Security

STEP 3 OF 6

3

Tighten the code-execution sandbox (time, memory, no network)

```
$ openclaw tools config exec --timeout 10s --memory 256mb --network deny
```

LAB 23

Security

STEP 4 OF 6

4

Confirm keys are not sitting in cleartext in the config

```
$ cat ~/.openclaw/config.toml | grep -i key
```

LAB 23

Security

STEP 5 OF 6

5

Review the audit log, filtering for blocked exec attempts

```
$ openclaw gateway logs --filter tool=exec
```

LAB 23

Security

STEP 6 OF 6

6

Export the audit log for weekly review

```
$ openclaw gateway logs --export ~/openclaw-audit-$(date +%F).log
```

LAB 23 · VERIFY

Security

✅ Test it

cat ~/.openclaw/config.toml does not contain raw API keys, a user not on the Telegram allowlist gets a not-authorized response, openclaw channel show telegram lists the deny list and messaging profile, and the audit log shows blocked exec attempts on a locked channel.

TOPIC 02 · HANDS-ON

Multi-Agent Workforce

LAB 24

In the OpenClaw capstone the learner splits the single agent into three cooperating agents — Sales (customer-facing on Telegram/WhatsApp), Research (supplier scouting with Firecrawl), and Ops (email, quotes, nightly crons) — each with least-privilege tools, and runs one end-to-end business scenario across all three, with a verified fallback using isolated OPENCLAW_HOME instances that cooperate via AgentMail and a shared

You'll build

A three-agent Sales/Research/Ops workforce that runs a customer order end-to-end into a real quote email

Tools: OpenClaw, openclaw agent (or isolated OPENCLAW_HOME instances), Firecrawl, AgentMail

LAB 24

Multi-Agent Workforce

STEP 1 OF 6

1

Create three named agents (or use isolated instances if unavailable)

```
$ openclaw agent create sales
```

LAB 24

Multi-Agent Workforce

STEP 2 OF 6

2

Give the Sales agent least-privilege tools (web + quoting, no shell/fs)

```
$ openclaw agent set sales --allow group:web --deny exec,fs.write,fs.delete
```

LAB 24

Multi-Agent Workforce

STEP 3 OF 6

3

Restrict Research to scraping only

```
$ openclaw agent set research --allow firecrawl,web_search,browser --deny exec,fs.write
```

LAB 24

Multi-Agent Workforce

STEP 4 OF 6

4

Attach the customer channels to the Sales agent

```
$ openclaw channel set telegram --agent sales
```

LAB 24

Multi-Agent Workforce

STEP 5 OF 6

5

Run a Research fallback instance with an isolated config dir

```
$ OPENCLAW_HOME=~/.openclaw-research openclaw onboard
```

LAB 24

Multi-Agent Workforce

STEP 6 OF 6

6

Give Ops the nightly consolidation cron

```
$ openclaw cron create --schedule "0 21 * * *" --prompt "Compile today's Nimbus Supplies orders and quotes sent, and list anything waiting on Research." --channel telegram --name ops-nightly-report
```

LAB 24 · VERIFY

Multi-Agent Workforce

✅ Test it

Three agents (or isolated instances) each have a distinct least-privilege tool set (Sales cannot exec, Research cannot write files, Ops is not on a public channel), a customer Telegram message produces a real quote email combining memory, the nimbus-quote skill, a Firecrawl price lookup and AgentMail delivery, and the nightly cron posts a summary referencing the same order.

TOPIC 02 · HANDS-ON

Dreaming — Idle Reflection & Self-Improvement

LAB 25

Enable OpenClaw's 'dreaming' feature so the Nimbus Supplies agent uses idle time to review its memory, consolidate what it has learned, and generate useful follow-up actions and skill improvements — then inspect what it produced. Reference the OpenClaw functions playlist for the walkthrough.

You'll build

An agent that, when idle, 'dreams' — reflecting on memory and proposing follow-ups you can review.

Tools: OpenClaw, dreaming, memory

LAB 25

Dreaming — Idle Reflection & Self-Improvement

STEP 1 OF 4

1

Enable the dreaming feature in OpenClaw settings/config

```
$ openclaw config set dreaming.enabled true
```

LAB 25

Dreaming — Idle Reflection & Self-Improvement

STEP 2 OF 4

2

Let the agent sit idle so a dream cycle runs (or trigger one)

```
$ openclaw dream run
```

LAB 25

Dreaming — Idle Reflection & Self-Improvement

STEP 3 OF 4

3

Review what the dream produced (insights, follow-ups, skill tweaks)

```
$ openclaw dream list
```

LAB 25

Dreaming — Idle Reflection & Self-Improvement

STEP 4 OF 4

4

Accept a useful follow-up the agent proposed

LAB 25 · VERIFY

Dreaming — Idle Reflection & Self-Improvement

✅ Test it

A dream cycle runs during idle time and produces reviewable insights/follow-ups from the agent's memory.

TOPIC 02 · HANDS-ON

Use Cases & Key Functions

LAB 26

Tie the OpenClaw track together with practical use cases for Nimbus Supplies — a Google Workspace CLI assistant, a data-analytics agent, and a social-media-marketing agent — combining channels, skills, tools, memory, dreaming and crons into end-to-end automations. Each use case has its own reference video.

You'll build

One or more end-to-end OpenClaw automations (Workspace, analytics or social media) solving a real use case.

Tools: OpenClaw, skills, tools, Google Workspace, crons, memory

LAB 26

Use Cases & Key Functions

STEP 1 OF 3

1

Pick a use case and compose the channels, skills and tools needed for it

LAB 26

Use Cases & Key Functions

STEP 2 OF 3

2

Run the automation end-to-end and verify the outcome

LAB 26

Use Cases & Key Functions

STEP 3 OF 3

3

Schedule it or hand it to the multi-agent team from Lab 24

LAB 26 · VERIFY

Use Cases & Key Functions

✅ Test it

A real use case (Google Workspace, data analytics or social media) runs end-to-end using OpenClaw's channels, skills, tools and crons.

TOPIC RECAP

Recap — OpenClaw — Automating a Business Back-Office

1. You can now: LO1: Install and configure an autonomous AI agent platform locally and on a container/VPS.
2. You can now: LO2: Connect an LLM model/provider to an agent via OAuth or API key.
3. You can now: LO2: Connect multiple messaging channels to one agent gateway.
4. You can now: LO3: Extend an agent with registry and custom skills.
5. You can now: LO3: Extend an agent with third-party tool integrations.
6. You can now: LO4: Operate an agent through its CLI and in-chat command surface.

03

TOPIC 03

Paperclip — Running & Governing a Company of AI Agents

Setup · Connect OpenAI Codex · Configure · Track Tasks · Automate Hiring · Setup Agents · Security · Assign Tasks

EXAM WEIGHTING 34%

Key Concepts — Paperclip — Running & Governing a Company of AI Agents

1. Paperclip is an operating system for a company of AI agents, self-hosted with Docker Compose; you are the Board, a CEO agent reports to you, and specialists report to the CEO.

2. You connect a coding model such as OpenAI Codex as the agents' engine, configure the company, and track AI tasks on a board: todo -> in_progress -> in_review -> done.

3. Hiring is automated behind human approval gates; all durable state lives in an embedded PostgreSQL database whose activity_log (tagged by actor_type) and cost_events tables give a CFO-style audit trail.

4. Security is enforced with budget caps (an 80% warning and a 100% pause rail) and approvals; you set up agents and assign tasks to deliver the company goal.

WHAT YOU'LL DO

Hands-On Labs — Paperclip — Running & Governing a Company of AI Agents

Labs 27–29

- Setup Paperclip
- Connect OpenAI Codex to Paperclip
- Configure Paperclip

Labs 30–32

- Track AI Tasks
- Automate AI Hiring
- Setup AI Agents

Labs 33–34

- Security
- Assign Task

TOPIC 03 · HANDS-ON

Setup Paperclip

LAB 27

Watch the Paperclip overview, then self-host Paperclip with Docker Compose and create the Nimbus Coffee company with a single goal and a monthly budget.

You'll build

A running Paperclip at http://localhost:3100 with a company (goal + budget).

Tools: Paperclip, Docker Compose

LAB 27

Setup Paperclip

STEP 1 OF 4

1

Clone the Paperclip repository

```
$ git clone https://github.com/paperclipai/paperclip.git
```

LAB 27

Setup Paperclip

STEP 2 OF 4

2

Start Paperclip with the quickstart compose file

```
$ docker compose -f docker-compose.quickstart.yml up --build
```

LAB 27

Setup Paperclip

STEP 3 OF 4

3

Open the dashboard

LAB 27

Setup Paperclip

STEP 4 OF 4

4

Create the company with a single goal and a monthly budget

LAB 27 · VERIFY

Setup Paperclip

✅ Test it

The dashboard loads at http://localhost:3100 and a company exists with a goal and budget.

TOPIC 03 · HANDS-ON

Connect OpenAI Codex to Paperclip

LAB 28

Give Paperclip's agents a brain by connecting the OpenAI Codex adapter, so the agents can reason and act. Confirm the adapter is detected and available.

You'll build

Paperclip using the OpenAI Codex adapter as the agents' model.

Tools: Paperclip, OpenAI Codex

LAB 28

Connect OpenAI Codex to Paperclip

STEP 1 OF 4

1

Install/log in to the OpenAI Codex CLI so Paperclip can detect it

LAB 28

Connect OpenAI Codex to Paperclip

STEP 2 OF 4

2

Open Settings -> Agents / Adapters in the dashboard

LAB 28

Connect OpenAI Codex to Paperclip

STEP 3 OF 4

3

Confirm the OpenAI Codex adapter shows as 'available'

LAB 28

Connect OpenAI Codex to Paperclip

STEP 4 OF 4

4

Restart Paperclip if the adapter is not detected

```
$ docker compose -f docker-compose.quickstart.yml restart
```

LAB 28 · VERIFY

Connect OpenAI Codex to Paperclip

✅ Test it

The OpenAI Codex adapter is detected and shows as available in Settings.

Configure Paperclip

LAB 29

Configure Nimbus Coffee: set the monthly budget, connect a workspace folder so agents write real files, and adjust company settings.

You'll build

A configured company with a budget and a connected workspace folder.

Tools: Paperclip, workspace

LAB 29

Configure Paperclip

STEP 1 OF 4

1

Open the company's settings in the dashboard

LAB 29

Configure Paperclip

STEP 2 OF 4

2

Set or adjust the monthly budget cap

LAB 29

Configure Paperclip

STEP 3 OF 4

3

Connect a workspace folder so agents create real deliverables

```
$ mkdir -p ~/paperclip-workspace/nimbus-coffee
```

LAB 29

Configure Paperclip

STEP 4 OF 4

4

Save the configuration and confirm it persists

LAB 29 · VERIFY

Configure Paperclip

✅ Test it

The company shows the configured budget and a connected workspace folder.

TOPIC 03 · HANDS-ON

Track AI Tasks

LAB 30

Use Paperclip's task board to track work through its four states — todo, in_progress, in_review, done — giving you visibility and control over what the agents are doing.

You'll build

A task board showing AI tasks moving across the four states.

Tools: Paperclip, task board

LAB 30

Track AI Tasks

STEP 1 OF 4

1

Open the task board for the company

LAB 30

Track AI Tasks

STEP 2 OF 4

2

Create or observe a task in the 'todo' column

LAB 30

Track AI Tasks

STEP 3 OF 4

3

Watch a task move todo -> in_progress -> in_review

LAB 30

Track AI Tasks

STEP 4 OF 4

4

Open a shell to review the task activity log

```
$ docker compose -f docker-compose.quickstart.yml exec paperclip sh
```

LAB 30 · VERIFY

Track AI Tasks

✅ Test it

Tasks are visible on the board and move through todo -> in_progress -> in_review -> done.

TOPIC 03 · HANDS-ON

Automate AI Hiring

LAB 31

Hire your CEO agent and let it propose specialist hires, each passing a human approval gate. This is how the company staffs itself automatically while you stay in control.

You'll build

An approved CEO agent and one or more approved specialist agents.

Tools: Paperclip, approval gates

LAB 31

Automate AI Hiring

STEP 1 OF 4

1

Hire the CEO agent from the company page

LAB 31

Automate AI Hiring

STEP 2 OF 4

2

Approve the CEO hire (you are the Board)

LAB 31

Automate AI Hiring

STEP 3 OF 4

3

Let the CEO propose specialist hires

LAB 31

Automate AI Hiring

STEP 4 OF 4

4

Review and approve each specialist at the approval gate

LAB 31 · VERIFY

Automate AI Hiring

✅ Test it

The CEO and approved specialists are active; each hire is logged as an actor_type=user decision.

TOPIC 03 · HANDS-ON

Setup AI Agents

LAB 32

Configure the hired agents — their mandate, tools and per-agent budget caps — so each specialist can carry out the delegated work for Nimbus Coffee.

You'll build

Configured agents with mandates, tools and per-agent budgets.

Tools: Paperclip, agents

LAB 32

Setup AI Agents

STEP 1 OF 4

1

Open an agent's configuration

LAB 32

Setup AI Agents

STEP 2 OF 4

2

Set the agent's mandate and the tools it may use

LAB 32

Setup AI Agents

STEP 3 OF 4

3

Set a per-agent budget cap

LAB 32

Setup AI Agents

STEP 4 OF 4

4

Save and confirm the agent is ready to receive tasks

LAB 32 · VERIFY

Setup AI Agents

✅ Test it

Each agent has a mandate, allowed tools and a budget cap, and is ready for tasks.

Security

LAB 33

Apply Paperclip's governance: company-wide and per-agent budget caps trigger an 80% warning and a 100% pause; approvals gate risky decisions; the audit trail records everything.

You'll build

Budget caps with the 80% warning and 100% pause rails demonstrated, plus an audit review.

Tools: Paperclip, budgets, PostgreSQL audit

LAB 33

Security

STEP 1 OF 4

1

Set a company and/or per-agent budget cap

LAB 33

Security

STEP 2 OF 4

2

Trigger the 80% warning and 100% pause, then resume

LAB 33

Security

STEP 3 OF 4

3

Audit actions and spend in the embedded PostgreSQL

```
$ psql "postgresql://postgres@localhost:5432/paperclip"
```

LAB 33

Security

STEP 4 OF 4

4

Separate human vs agent actions with actor_type

```
$ SELECT actor_type, COUNT(*) FROM activity_log GROUP BY actor_type;
```

LAB 33 · VERIFY

Security

✅ Test it

The 80% warning and 100% pause fire, resume works, and the audit shows actions by actor_type and total spend.

Assign Task

LAB 34

Delegate real work: assign tasks to specialists, watch them execute and produce deliverables in the workspace, and review and accept the results to complete the Nimbus Coffee goal.

You'll build

Assigned tasks executed by agents, producing real deliverables you review and accept.

Tools: Paperclip, task board, workspace

LAB 34

Assign Task

STEP 1 OF 4

1

Assign a task to a specialist agent

LAB 34

Assign Task

STEP 2 OF 4

2

Watch the agent execute and write output to the workspace

```
$ ls -R ~/paperclip-workspace/nimbus-coffee
```

LAB 34

Assign Task

STEP 3 OF 4

3

Review the deliverable in the in_review state

LAB 34

Assign Task

STEP 4 OF 4

4

Approve to move it to done, or send it back for revision

LAB 34 · VERIFY

Assign Task

✅ Test it

An assigned task is executed by an agent, produces a real file, and is reviewed and accepted to done.

TOPIC RECAP

Recap — Paperclip — Running & Governing a Company of AI Agents

1. You can now: LO1: Install and self-host Paperclip, then found your company.
2. You can now: LO2: Connect a coding model (OpenAI Codex) as the agents' engine.
3. You can now: LO2: Configure the company — settings, budget and workspace.
4. You can now: LO3: Track AI work on the task board.
5. You can now: LO3: Automate hiring of the CEO and specialist agents behind approval gates.
6. You can now: LO2: Set up and configure the AI agents and their capabilities.

WRAP-UP

Course Summary & Next Steps

LEARNING OUTCOMES

What You Achieved

1

Analyzed AI Applications

Analyse AI (LLM-based) applications across a range of industries to identify their capabilities and limitations.

2

Design & Chatbot Efficiency

Establish the relationship between AI algorithm/agent design and chatbot/agent efficiency.

3

Evaluated & Improved RAG

Evaluate and improve the effectiveness of AI (RAG and agent) applications in product development.

NEXT STEPS

Continuing Your AI-Agent Journey

1. Deploy an agent on a VPS or cloud backend for round-the-clock, 24/7 operation.
2. Extend the labs with your own skills, tools and integrations for a workflow you care about.
3. Apply governance in production — budgets, approvals and an audit trail.
4. Explore multi-agent orchestration to deliver real end-to-end business workflows.

Assessment

1. Written Assessment (SAQ) — 1 hour.
   Practical Performance (PP) — 1 hour.
2. Open book: slides, Learner Guide and approved materials only.
3. Remember to take the Assessment digital attendance (TRAQOM).
4. Submit your completed answers on the LMS at https://lms-tms.tertiaryinfotech.com/.

ON ASSESSMENT DAY

Assessment Flow

1. TRAQOM survey — scan the QR code on the LMS
2. Assessment digital attendance — scan the SSG QR
3. Sit WA (SAQ) then PP — open book
4. Submit your answers on the LMS
5. Sign the Assessment Summary Record

Digital Attendance (Mandatory)

1. It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
2. The trainer/administrator displays the digital attendance QR code from the SSG portal.
3. Scan the QR code with your mobile phone camera and submit your attendance.
4. A minimum of 75% attendance is required to be eligible for assessment and funding.

BUILD YOUR AI WORKFORCE

Thank You!

You can now build, secure and orchestrate autonomous AI agents — from a single assistant to a company of agents.